

Progetto di High Performance Computing 2024/2025

Davide Marchetti, matr. 0001021628

07/09/2025

1 Introduzione

In questo progetto vengono presentate due possibili soluzioni parallele di un algoritmo che calcola lo *skyline* di un insieme di punti.

Considerando un insieme di N punti in uno spazio a D dimensioni, si dice che un punto P_i domina P_j se sono vere le seguenti condizioni:

- per ogni dimensione $k = 0, \dots, D - 1$ allora $P_{i,k} \geq P_{j,k}$
- esiste almeno una coordinata k per cui la disuguaglianza al punto precedente valga in senso stretto

L'insieme dei punti che non sono dominati da nessun altro punto viene chiamato *skyline* $Sk(P)$. Lo pseudocodice dell'algoritmo utilizzato per calcolare lo *skyline* è il seguente:

```
1: procedure SKYLINE( $P_0, P_1, \dots, P_{N-1}$ )
2:   bool  $S[0 \dots N - 1]$ 
3:   integer  $i, j$ 
4:   for  $i = 0$  to  $N - 1$  do
5:     if  $S[i]$  then
6:       for  $j = 0$  to  $N - 1$  do
7:         if  $S[j]$  and  $P_i$  domina  $P_j$  then
8:            $S[j] \leftarrow \text{false}$ 
9:            $r - -$ 
10:           $its + +$ 
11:        end if
12:      end for
13:    end if
14:  end for
15:  for  $i = 0$  to  $N - 1$  do
16:    if  $S[i]$  then
17:      print  $P_i$ 
18:    end if
19:  end for
20: end procedure
```

In particolare confronta ogni punto dell'insieme con tutti gli altri eliminando dalla selezione tutti i punti dominati dal punto preso in considerazione nell' i -esima iterazione.

2 Versione OpenMP

Inizialmente è necessario inizializzare il vettore $S[0 \dots N - 1]$ assegnando ad ogni elemento il valore 1.

```
for  $i = 0$  to  $N - 1$  do
     $S[i] = 1$ 
end for
```

In questo ciclo non sono presenti dipendenze quindi è possibile parallelizzarlo banalmente, utilizzando il pattern di programmazione parallela chiamato *Embarrassingly Parallel*; così facendo ogni *thread* esegue la stessa operazione ma ognuno su un gruppo di elementi differenti.

Nei due cicli successivi vengono incrementati due valori di due variabili differenti, se queste operazioni vengono eseguite in maniera parallela si verificano delle *race condition*, in quanto p *thread* accedono allo stesso indirizzo di memoria modificando il valore all'interno e contemporaneamente uno o più *thread* potrebbero leggere lo stesso valore.

Per risolvere questo problema è possibile eseguire le operazioni in maniera atomica, in modo tale che soltanto un *thread* accedeva al contenuto della variabile modificandone il valore. Questa soluzione non è ottimale quando N è particolarmente grande; per questo motivo viene utilizzato il pattern di programmazione parallela *Reduce*: ogni *thread* crea una copia privata della variabile in questione per poi eseguire l'operazione binaria su di essa. Nel momento in cui l'ultimo *thread* conclude le operazioni, ogni valore delle p variabili private viene sottoposto di nuovo all'operazione binaria memorizzando il risultato in una sola variabile. Questo meccanismo in OpenMP può essere utilizzato tramite la direttiva `reduction(operator: variabile)` specificando l'operazione binaria da effettuare e la variabile su cui vengono effettuate le operazioni; effettuando una *reduction* sulle variabili `its` e `r`, con rispettivamente gli operatori di somma e differenza, è possibile eseguire le operazioni in parallelo.

Così facendo è possibile parallelizzare soltanto il ciclo interno essendo presente una dipendenza sugli indici dei due cicli: supponiamo di voler parallelizzare il ciclo esterno, tutti i processi hanno l'indice i diverso, però eseguendo la prima iterazione del ciclo, dove si ha $j = 0$, se entrambi i punti i dominano il punto j , allora nel caso peggiore tutti i processi eseguono le due operazioni di incremento e decremento.

Tuttavia parallelizzando il ciclo interno significa effettuare N iterazioni del ciclo esterno in maniera seriale definendo ogni volta una nuova regione parallela; nuovamente questo

approccio non è molto efficiente nel caso in cui N sia molto grande e non rende scalabile il programma. Il modo migliore per parallelizzare i due cicli consiste nel parallelizzare solamente il ciclo esterno apportando delle specifiche modifiche per rimuovere la dipendenza fra i due indici.

In particolare viene modificato lo *scope* del vettore s in modo tale da creare una copia del vettore in ogni *thread* con tutti i valori inizializzati in precedenza. Per far ciò OpenMP permette di definire lo *scope* del vettore utilizzando la direttiva `firstprivate(s)`, in questo modo ogni *thread* possiede una copia del vettore originale già inizializzata. Inoltre è necessario modificare l'algoritmo e farlo funzionare in maniera diversa: ogni *thread* quando analizza il punto i -esimo se trova un punto j che domina il punto i allora elimina il punto i dalla selezione (l'opposto della versione originale) e smette di eseguire ulteriori confronti passando al punto i successivo. Applicando le precedenti modifiche si ottiene il seguente algoritmo:

```

1: #pragma omp parallel for default(shared) \
2:   firstprivate(s) reduction(-: r) reduction(+: its)
3: for  $i = 0$  to  $N - 1$  do
4:   for  $j = 0$  to  $N - 1$  do
5:     if  $S[i]$  and  $S[j]$  and  $i \neq j$  and  $P_j$  domina  $P_i$  then
6:        $S[i] \leftarrow \text{false}$ 
7:        $r \leftarrow -$ 
8:        $its \leftarrow +$ 
9:        $j = N$ 
10:    end if
11:  end for
12: end for

```

2.1 Prestazioni

Durante l'esecuzione dei test si osserva che il primo di essi (`circle-N1000-D2.in`), non viene eseguito efficientemente a causa della dimensione piccola dell'insieme dei punti e dal numero di punti stesso. Invece, i test successivi hanno ottenuto buoni risultati: uno di essi raggiunge uno *speedup* superlineare grazie alla piccola dimensione dei punti.

Test	1 Core	12 Core	Speedup	Efficienza
circle	0,000457	0,008241	0,055457	0,004621
test1	2,019119	0,164357	12,284929	1,023744
test2	9,475858	0,885412	10,702204	0,891850
test3	39,283653	3,745326	10,488713	0,874059
test4	25,179834	2,516230	10,006966	0,833913
test5	161,159298	14,613687	11,027969	0,918997
test6	242,703979	26,078162	9,306789	0,775565
test7	282,882128	38,253679	7,394899	0,616241
worst	128,839396	11,653567	11,055790	0,921315

3 Versione CUDA

Per quanto riguarda la versione con CUDA sono presenti le stesse dipendenze e problematiche riscontrate nella versione OpenMP, ma vengono affrontate in maniera diversa. In questa versione vengono invocati due *kernel* diversi: uno viene utilizzato per inizializzare il vettore S e uno per applicare l'operatore *skyline*. Il numero di *thread* utilizzati da entrambi i *kernel* è costante a differenza del numero di blocchi che vengono calcolati con la seguente formula:

$$\# \text{ blocchi} = \frac{N + BLOCKDIM - 1}{BLOCKDIM} \quad (1)$$

In questo modo anche se il numero di punti non è multiplo di 1024, numero di *thread* per blocco (BLOCKDIM), vengono utilizzati un numero sufficiente di blocchi tale che il numero di *thread* totale sia sempre maggiore del numero di punti.

Ogni *kernel* utilizza lo stesso principio di funzionamento: viene assegnato ad ogni *thread* un indice che verrà utilizzato per accedere all'elemento del vettore S e all'elemento del vettore *points*. Per il calcolo dell'indice viene utilizzata la seguente formula:

$$\text{elem} = \text{threadIdx.x} + \text{blockIdx.x} \times BLOCKDIM \quad (2)$$

Di conseguenza, se N non è multiplo di BLOCKDIM, verranno utilizzati più *thread* rispetto al numero di punti in totale, perciò una serie di indici punterà a zone di memoria al di fuori dei limiti del vettore contenente i punti. Siccome una parte dei *thread* non dovrà eseguire alcuna operazione, dopo aver calcolato l'indice, si effettua un confronto con il numero di punti in totale, se l'indice è maggiore di quest'ultimo allora quel *thread* non esegue le operazioni.

```

for ( $i = 0$ ;  $i < N$  and  $S[\text{elem}]$ ;  $i++$ ) do
  if  $P_i$  domina  $P_{\text{elem}}$  then
     $S[\text{elem}] \leftarrow \text{false}$ 
     $s\_its[tindex] = 1$ 
    break
  end if
end for

```

Una volta eseguito il controllo sull'indice ogni *thread* confronta ogni punto dell'insieme di partenza con il punto a loro assegnato, identificato dall'indice **elem** calcolato in precedenza. Se il punto i -esimo domina il punto analizzato dal *thread*, viene modificato il valore in posizione **tindex** (l'id del *thread*) nel vettore allocato nella memoria condivisa di ogni blocco.

Il vettore **s_its** ha un numero di elementi pari al numero di *thread* allocati in ogni blocco, viene utilizzato per calcolare il numero di iterazioni che effettua l'algoritmo e di conseguenza anche il numero di punti dello *skyline*. In particolare se il punto dell' n -esimo *thread* è dominato dal punto i -esimo allora il valore del vettore in posizione n viene impostato a 1. Soltanto dopo una sincronizzazione dei *thread* di un blocco uno di essi somma tutti i valori di **s_its** per ottenere il numero di iterazioni totali eseguite da un blocco. Infine, per ottenere il risultato finale, vengono sommati in maniera atomica gli ultimi valori calcolati da ogni blocco, ed il risultato viene salvato in una variabile globale in modo da poterlo copiare dal *device* all'*host*.

```

if tindex == 0 then
   $local\_its = 0$ 
  for  $i = 0$  to  $BLOCKDIM$  do
     $local\_its += s\_its[i]$ 
  end for
   $atomicAdd(d\_its, local\_its)$ 
end if

```

3.1 Gestione della memoria

Per quanto riguarda la memoria vengono allocati sulla GPU due principali vettori: il vettore contenente tutti i punti dell'insieme, chiamato **d_points**, con un numero di elementi pari a $N \times D$ dove N è il numero di punti e D è la dimensione di un punto, e il vettore S di dimensione N .

In seguito vengono copiati soltanto i valori del vettore **points**, ovvero il vettore di punti memorizzato sull'*host*, in quanto il vettore S verrà inizializzato tramite l'esecuzione di

un *kernel* prima di eseguire la computazione principale. Vengono salvati sulla GPU il numero di elementi N e la dimensione di ogni punto D come costanti, non essendo modificati durante l'esecuzione del programma.

Per ogni blocco viene allocato sulla memoria condivisa un vettore di dimensione `BLOCKDIM`, dove ogni elemento specifica se il punto analizzato dal *thread* n -esimo è dominato da un punto o no. Così facendo si può ottenere il numero di punti analizzati da ogni singolo blocco che appartengono allo *skyline*.

3.2 Prestazioni

Durante l'analisi delle prestazioni non vengono presi in considerazione sia il tempo impiegato dalla GPU ad allocare la memoria, sia il tempo impiegato dalla CPU a copiare i valori sulla memoria della GPU. Per misurare le prestazioni del modello si calcola il *throughput*, ovvero il numero di operazioni effettuate da tutti i *core* in un secondo e si misura in FLOPS (*FL*oating *P*oint *O*perations *P*er *S*econds).

Test	Time (s)	Operazioni (mln)	GFLOPS
circle	0,000475	2	4,214
test1	0,061196	30000	490,225
test2	0,171605	40000	233,093
test3	0,380716	100040,004	262,768
test4	0,303042	80014,40065	264,038
test5	0,994650	200000	201,078
test6	2,414202	501000,5	207,522
test7	4,017472	2016032	501,816
worst	0,541381	100000	184,713

4 Conclusioni

Le versioni proposte non utilizzeranno l'algoritmo più efficiente, nonostante ciò hanno ottenuto degli ottimi risultati. In particolare la versione in OpenMP offre una buona scalabilità con insiemi di punti con dimensione non molto elevata ma allo stesso tempo offre delle ottime prestazioni nei casi peggiori, ovvero quando tutti i punti dell'insieme appartengono allo *skyline*. Nella versione CUDA non viene calcolato il numero di punti appartenenti allo *skyline* all'interno del *kernel* ma si ricava dal numero di iterazioni che vengono effettuate sottraendole al numero totale di punti.